

データ構造 区間クエリ Wavelet Matrix Binary Trie

Wavelet Matrix (基礎)

[解説動画はこちら](#)です。

1. 概要

本講座では、**Wavelet Matrix** というデータ構造について解説します。

Wavelet Matrix とは大雑把に言えば、静的な非負整数列 $A = (A_0, A_1, \dots, A_{N-1})$ に対して、クエリ (L, R) が与えられるたびに、「多重集合 $\{A_L, A_{L+1}, \dots, A_{R-1}\}$ に対応する Binary Trie」に対する探索を行うことができるデータ構造です。応用範囲は広いですが、本講座では次の 2 つの問題に限定して解説します。

- $A_L, A_{L+1}, \dots, A_{R-1}$ のうちで昇順 k 番目の要素を求める。
- $A_L, A_{L+1}, \dots, A_{R-1}$ のうちで a 以上 b 未満の要素の個数を求める。

本講座では Wavelet Matrix について、その前身である Wavelet Tree についても触れ、特にアイデアの分かりやすさを重視して解説しています。

Wavelet Matrix は仕組みを理解すれば、この 2 つの問題以外に応用することも難しくありません。どのような問題に応用できるかについては改めて、[Wavelet Matrix \(発展\)](#) で整理することにします。

2. 前提知識

本講座では多くの部分で、Binary Trie の理解に基づいた解説をしているので、Binary Trie を理解していることが望ましいです。ただし、はじめに復習をしているので、木構造として表

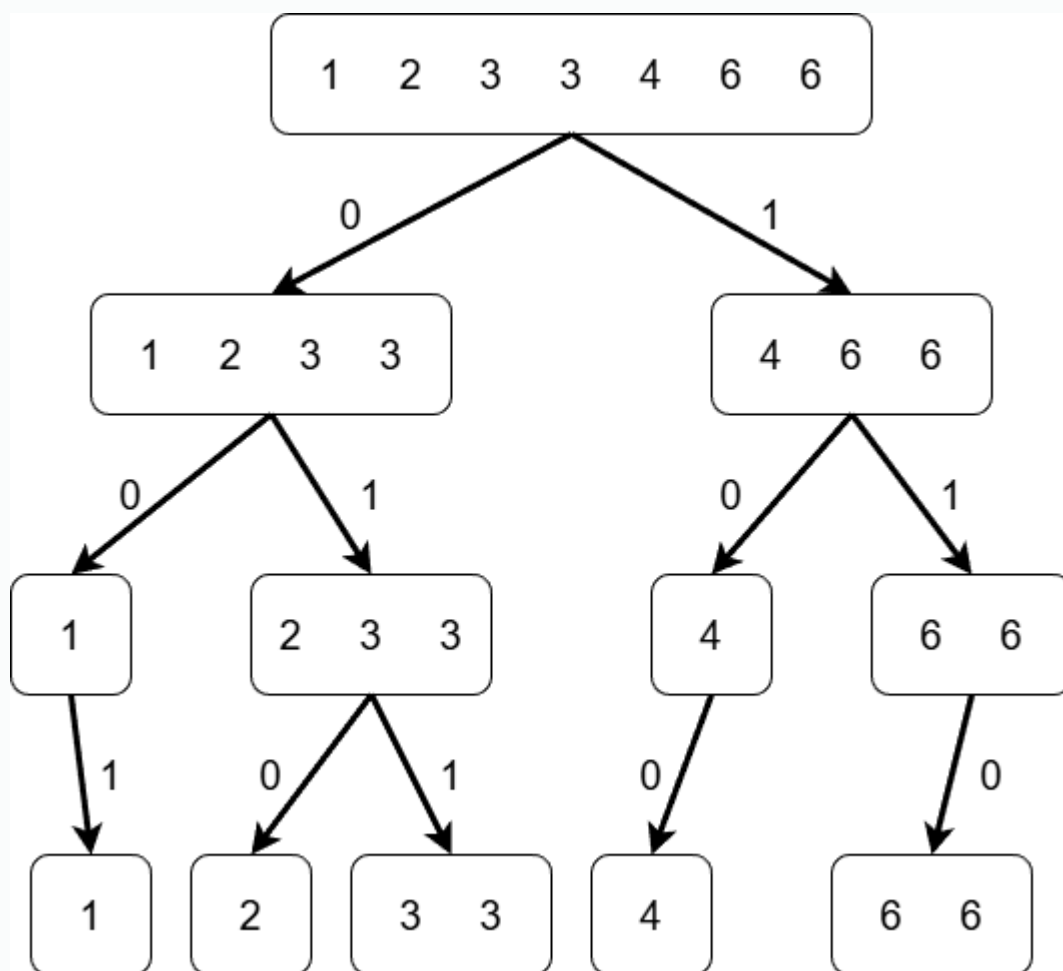
されるデータ構造に慣れていれば直接 Binary Trie を知らなくても理解は可能です。

3. Binary Trie の復習

3.1. Binary Trie

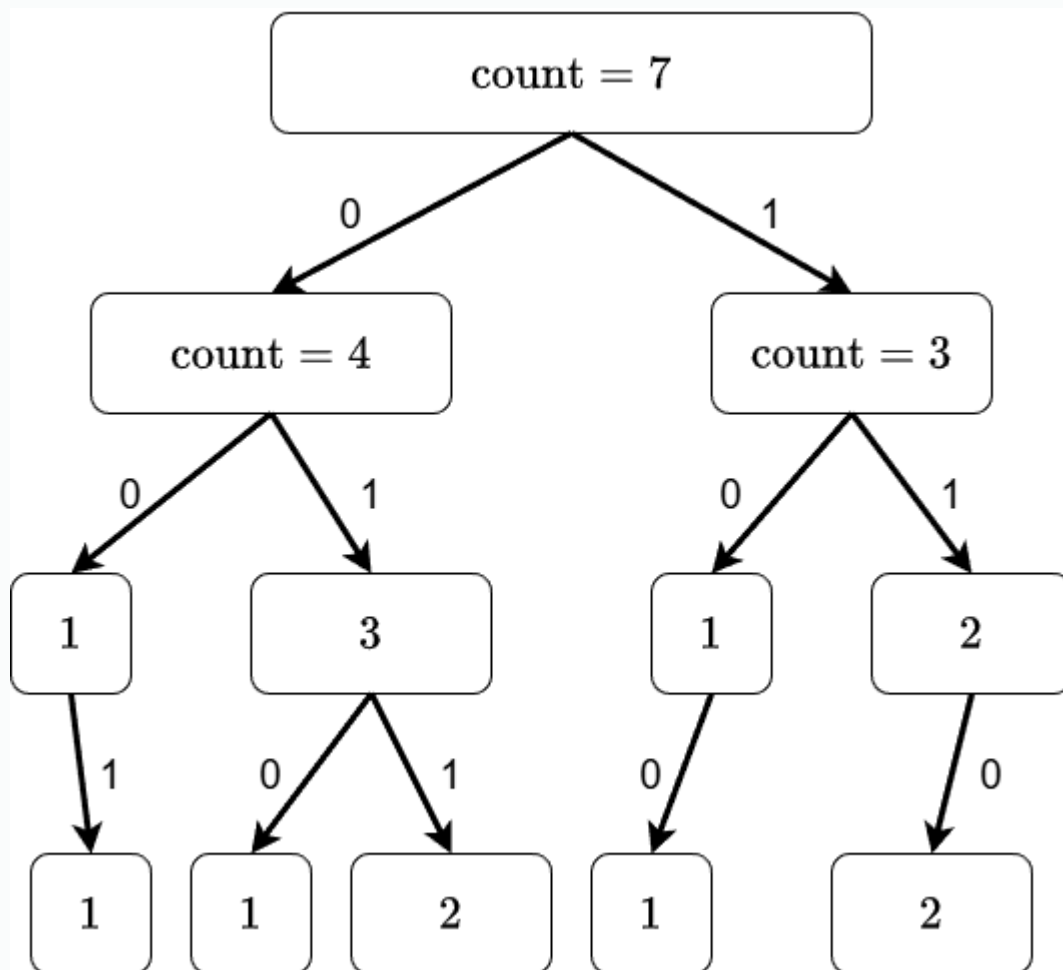
Wavelet Matrix の仕組みやできることを理解する上では、まず **Binary Trie** について理解することが前提となります。本講座では Binary Trie の具体的な実装にまでは踏み込みませんが、まずは Binary Trie がどのようなデータ構造であったかについて簡単に復習します。

高さ H の Binary Trie は、 0 以上 $2^H - 1$ 以下の整数をその 2 進法表記によって分類していく過程を表す二分木構造です。ここではより詳しい解説は省略しますが、ひとつの例を図示します。例えば次の図は多重集合 $\{1, 2, 3, 3, 4, 6, 6\}$ に対応する Binary Trie です。



各ノードは、ある半開区間 $[L, R)$ (つまり集合 $\{x \in \mathbb{Z} \mid L \leq x < R\}$) に対応します。さらに高さ h のノードについて、 L, R はある i によって $L = i \cdot 2^h$, $R = (i + 1) \cdot 2^h$ と書くことができます。

上の図では説明のため、各ノードに対して多重集合を書き込んでいますが、実際にはこれらの値はノードに持たせることは通常せず、要素の個数などの目的に応じた集約値を持たせることになります。各ノードに要素の個数を持たせた場合、実装される値は次のようになります。



各ノードがどのような整数区間に対応するかは、直接は保持せずとも、根からそのノードまでどのような経路を辿ったかによって分かります。

以下では多重集合に対応する Binary Trie が構築されているとして、「昇順 k 番目の値」および「 a 以上 b 未満の要素の個数」を求める方法について確認します。

3.2. 昇順 k 番目の要素

ここでは k 番目について、最小値を 0 番目と数える **0-based index** で説明します。

多重集合に対応する Binary Trie が構築されているとき、その多重集合での昇順 k 番目の要素を $O(H)$ 時間で求めることができます。葉ではないノードにおいて、そのノードに対応する多重集合の昇順 k 番目の値を求める問題を考えて、次のようにして Binary Trie を探索すればよいです。

- 左部分木内の要素数 x を調べる.
- $k < x$ ならば, 左部分木に進み k 番目の要素を求める.
- $x \leq k$ ならば, 右部分木に進み $(k - x)$ 番目の要素を求める.

3.3. a 以上 b 未満の要素の個数

多重集合に対応する Binary Trie が構築されているとき, その多重集合での a 以上 b 未満の要素の個数を $O(H)$ 時間で求めることができます.

これは「 b 未満の要素の個数」から「 a 未満の要素の個数」を引いたものなので, 「 a 未満の要素の個数」を求める方法が分かればよいことになります.

現在調べているノードに対応する非負整数区間 $[l, r)$ を保持しながら木を探索し, 葉ではないノードにおいて,

- 左右の部分木の境界 $m = (l + r)/2$ を求める.
- $a \leq m$ ならば, 左部分木に進む.
- $m < a$ ならば, 左部分木の要素数を答に加えて右部分木に進む.

という処理をすればよいです.

4. Wavelet Tree (概説)

$A = (A_0, A_1, \dots, A_{N-1})$ を, 0 以上 $2^H - 1$ 以下の整数からなる静的な列 (つまりクエリによって変わることがない列) だとします. 本節で解説する **Wavelet Tree**, および第 5 節で解説する **Wavelet Matrix** は, 大まかにいえば, 次のことが行えるデータ構造です.

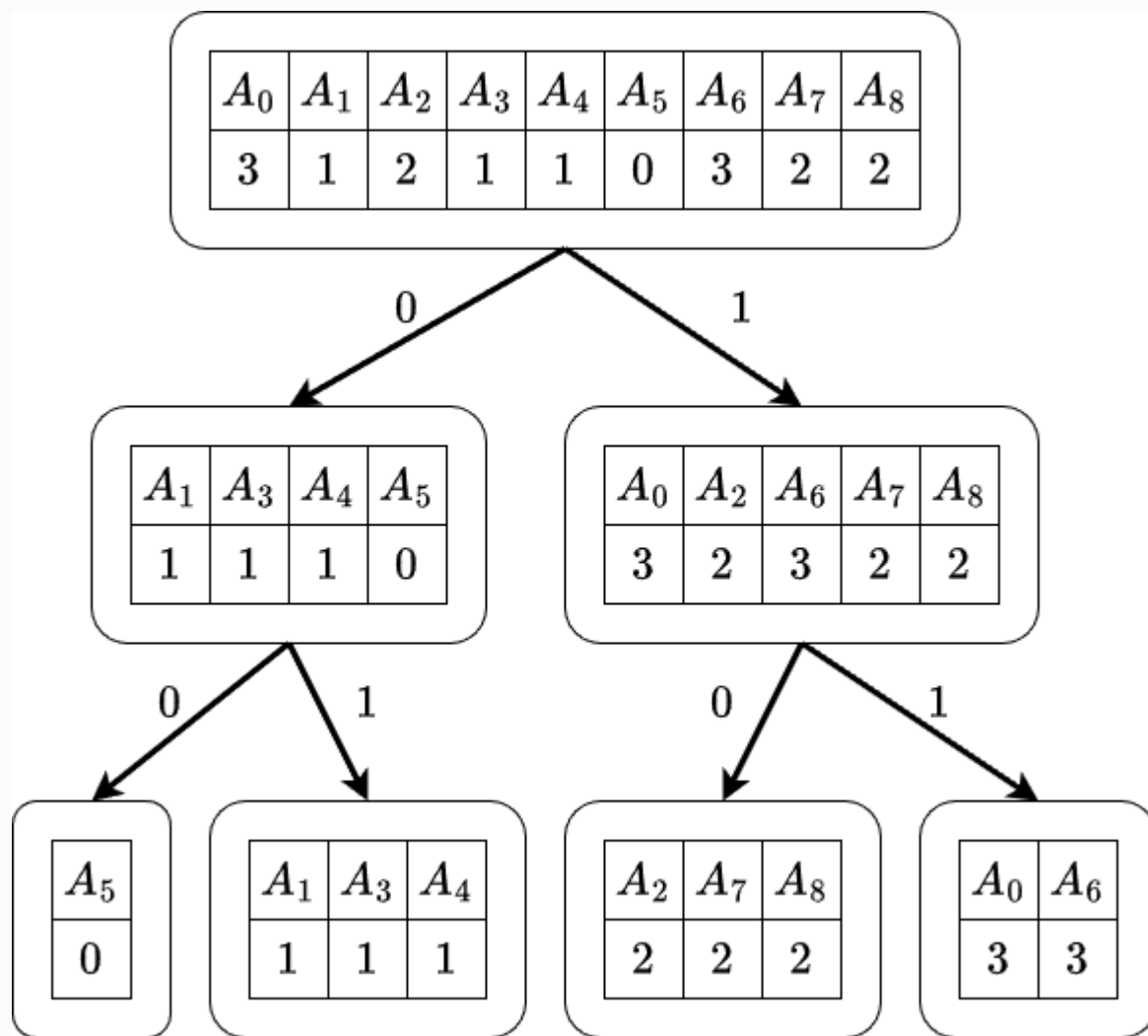
- 区間クエリ $[L, R)$ が与えられたとき, 「多重集合 $\{A_L, A_{L+1}, \dots, A_{R-1}\}$ に対応する Binary Trie」 (以下, **区間 $[L, R)$ の Binary Trie** と呼びます) を扱うことができる.

このことを実現するためのアイデアが比較的分かりやすい, Wavelet Tree について, 具体例を図示することによって解説します.

$$A = (A_0, A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8) = (3, 1, 2, 1, 1, 0, 3, 2, 2)$$

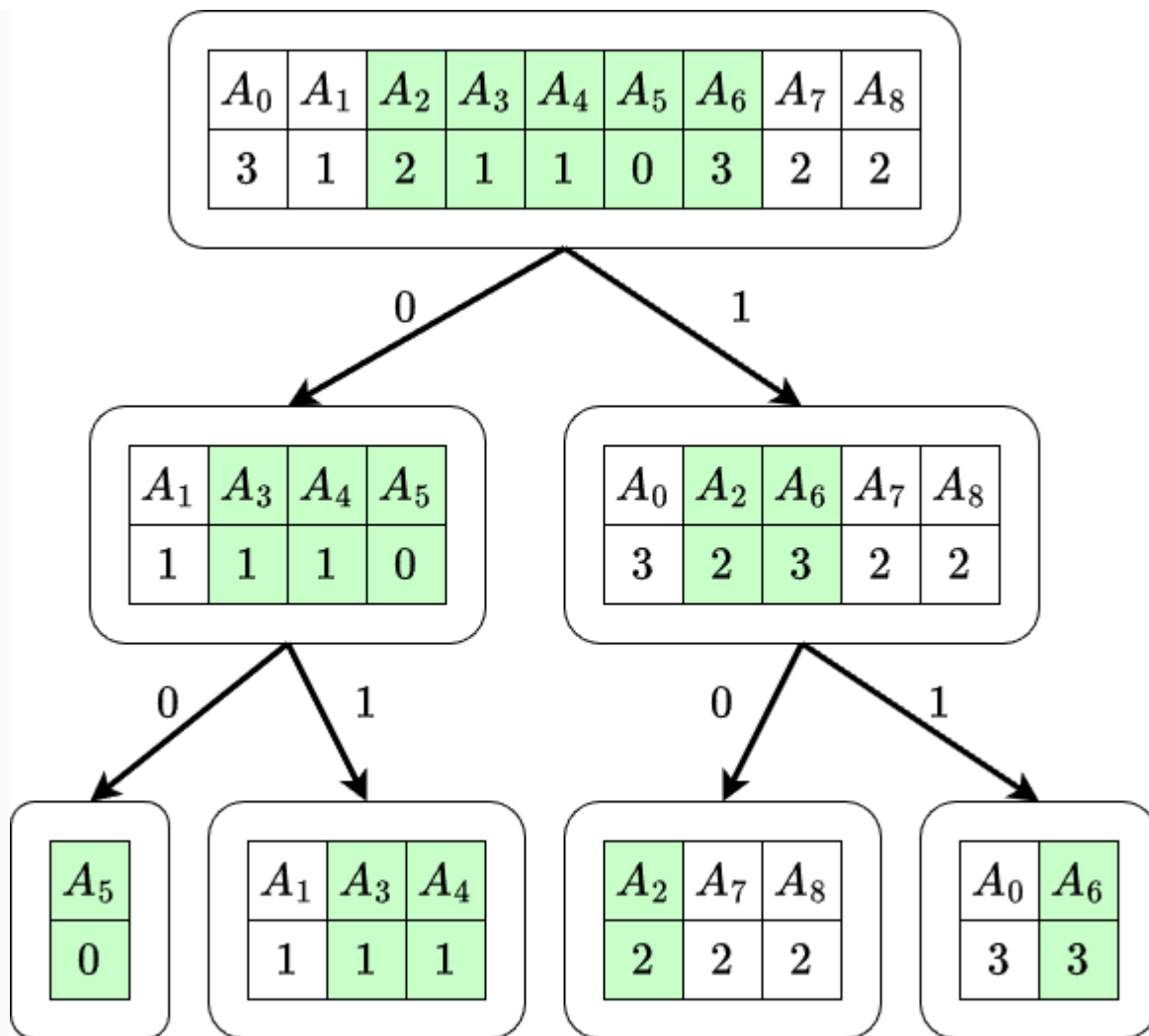
であるとして, A に対応する Binary Trie を構築しましょう. ただし, 要素は A におけるインデックスも含めて扱い, 各ノードにおいて**要素をインデックス昇順に並べる**ことにしま

す.

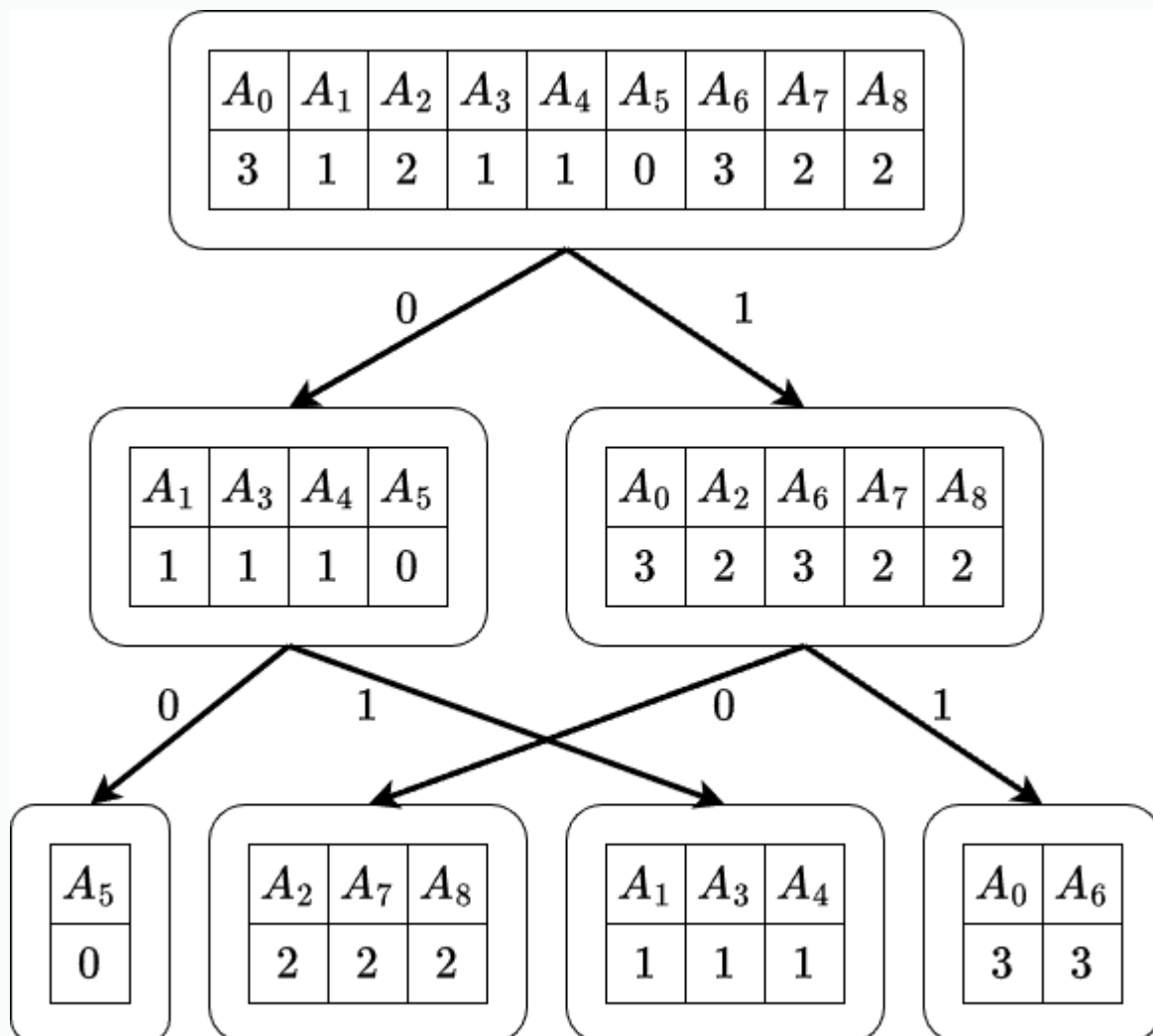


大雑把な説明ではありますが、これが A に対する Wavelet Tree です.

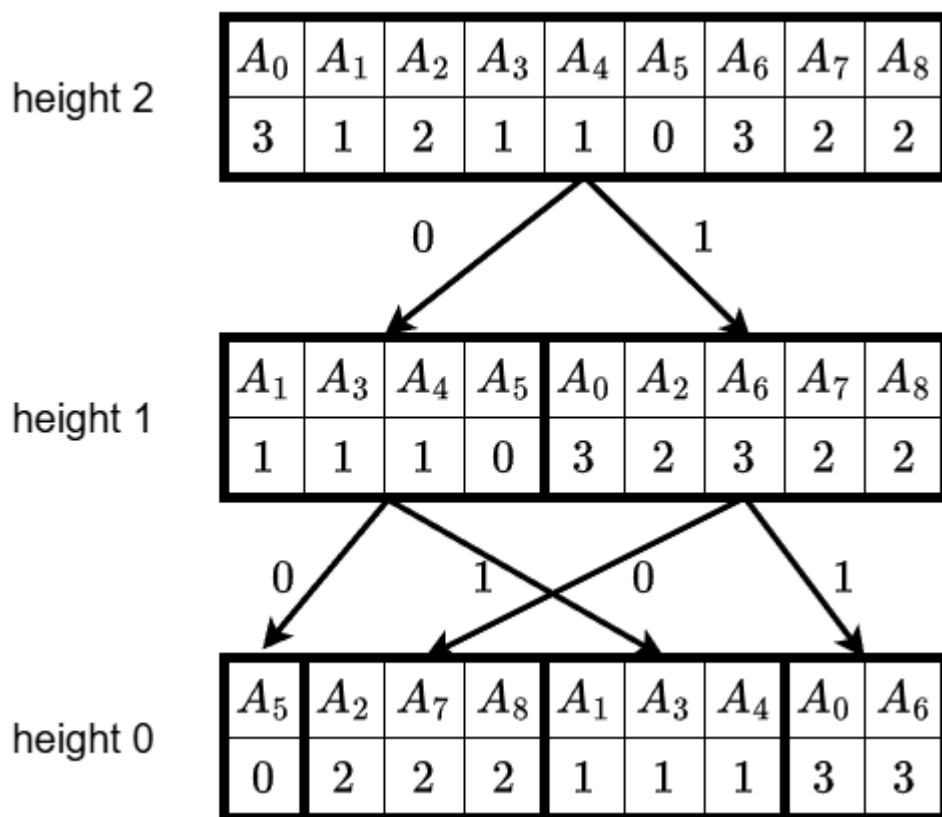
区間 $[L, R)$ の Binary Trie を扱えるとはどういうことでしょうか？例えば $L = 2, R = 7$ の場合を図示すると次の図のようになります.



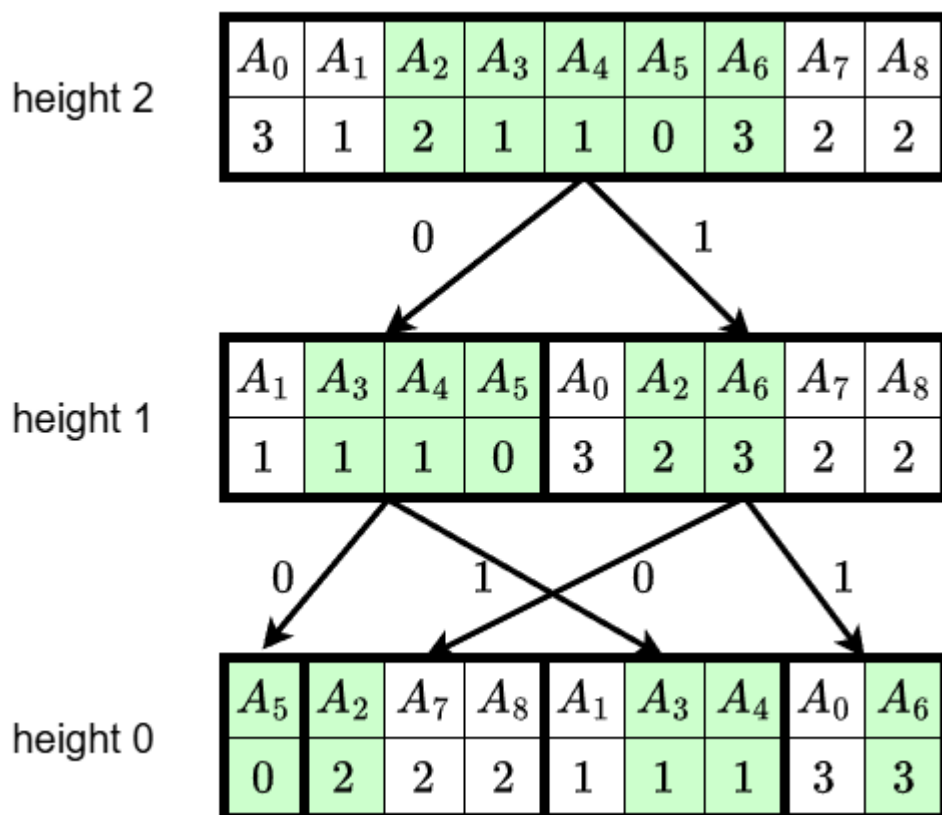
Wavelet Tree のノードの並び順を変えて、各高さごとに、**直前の高さの左部分木全体を左側、右部分木全体を右側に並べる**という順序で並べます（左側同士、右側同士では直前の高さでの順序を保ちます）。



さらに、同じ高さのノードを結合してひとつの列として表します。



大雑把に言えば、これが **Wavelet Matrix** です。やはり列全体を区間 $[L, R)$ に制限すると、それにもなって各ノード内に格納されている要素全体も何らかの区間に制限されます (Wavelet Tree のときと比べてノードの並び順が変わっただけなので、当然です)。



5.2. 実際に実装するデータ

5.1 節のアイデアをもとに、実際に区間 $[L, R)$ の Binary Trie を扱うためには、どのようなデータがあるとよいでしょうか？

まず、各 h ($0 \leq h \leq H - 1$) について、高さ $h + 1$ の層から高さ h の層に向かって、それぞれの要素が左部分木、右部分木のどちらに移動したのかを表す 0, 1 列を用意しましょう。

height 2	A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	A_8
	3	1	2	1	1	0	3	2	2
移動方向	1	0	1	0	0	0	1	1	1

height 1	A_1	A_3	A_4	A_5	A_0	A_2	A_6	A_7	A_8
	1	1	1	0	3	2	3	2	2
移動方向	1	1	1	0	1	0	1	0	0

height 0	A_5	A_2	A_7	A_8	A_1	A_3	A_4	A_0	A_6
	0	2	2	2	1	1	1	3	3

実はこれらの 0, 1 列は必要な情報をすべて持っています。

各高さのどの位置にノードの境界があるか、何番目にどの A_i が来るか、その A_i の値は何であるか（上図で灰色としている情報）は（計算量を問わなければ）すべてこれらの 0, 1 列から復元可能です。特に、各ノードに対応する整数区間や、高さ 0 における A_i の値は、根からそのノードまでどのような経路を辿ったかによって分かる（Binary Trie の場合と同様）ことに注意して、確認してください。

この高さごとの 0, 1 列（にいくつかの付加情報を加えたもの）が Wavelet Matrix です。

Wavelet Matrix

height 2→1	1	0	1	0	0	0	1	1	1
height 1→0	1	1	1	0	1	0	1	0	0

実際には、これらの 0, 1 列の累積和を実装することで、区間内にある 0, 1 の個数が $O(1)$ 時間で取得できるようにします。次は Wavelet Matrix を構築する実装です。

```
struct Wavelet_Matrix {
    int H, N;
    vector<vector<int>> dat;
    Wavelet_Matrix(int H, vector<int> A) : H(H), N(A.size()), dat(H) {
        for (int h = H - 1; h >= 0; --h) {
            // height h+1 to h.
            vector<int> dir(N);
            vector<int> left, right;
            for (int i = 0; i < N; ++i) {
                dir[i] = A[i] >> h & 1;
                (dir[i] == 0 ? left : right).emplace_back(A[i]);
            }
            int a = left.size(), b = right.size();
            for (int i = 0; i < a; ++i) A[i] = left[i];
            for (int i = 0; i < b; ++i) A[a + i] = right[i];

            // dat[h] := prefix sum of dir
            dat[h].resize(N + 1);
            for (int i = 0; i < N; ++i) dat[h][i + 1] = dat[h][i] + dir[i];
        }
    }
};
```

Wavelet Matrix の構築の計算量は $O(NH)$ 時間です。

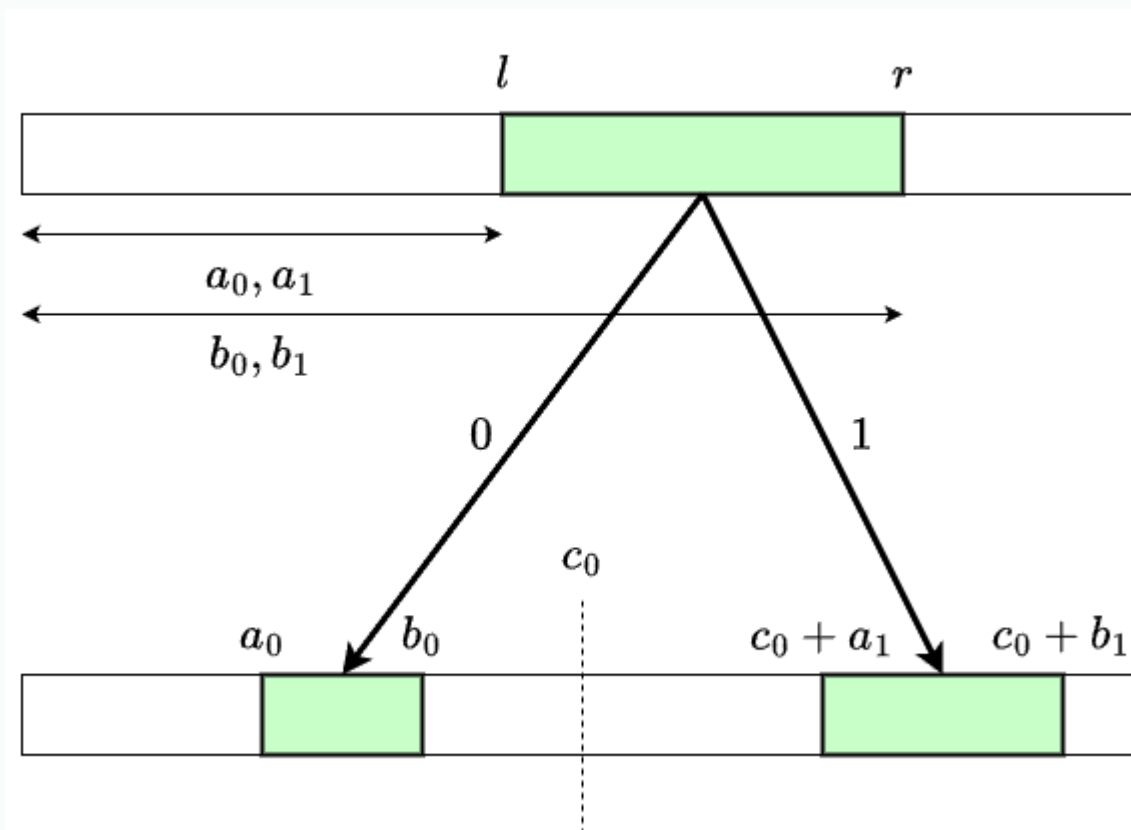
5.3. Wavelet Matrix における探索

以上の準備のもと、Wavelet Matrix を利用して、「区間 $[L, R)$ の Binary Trie における探索」を行えます。

現在調べているノードが高さ $h + 1$ の区間 $[l, r)$ として表されているとします。高さ $h + 1 \rightarrow h$ の $0, 1$ 列において、

- $[0, l)$ 内にある $0, 1$ の個数を a_0, a_1
- $[0, r)$ 内にある $0, 1$ の個数を b_0, b_1
- $[0, N)$ 内にある 0 の個数を c_0

とすれば、左部分木は高さ h の区間 $[a_0, b_0)$ 、右部分木は高さ h の区間 $[c_0 + a_1, c_0 + b_1)$ として表されます。



```
tuple<int, int, int, int> get_subtree_range(int h, int l, int r) {  
    // left and right subtree of (h+1, l, r)  
    int a0 = l - dat[h][l], a1 = dat[h][l];  
    int b0 = r - dat[h][r], b1 = dat[h][r];  
    int c0 = N - dat[h][N];  
    return {a0, b0, c0 + a1, c0 + b1};  
}
```

特に、左右の部分木の要素数を知ること、左右の部分木に進むこともできるため、高さ H の区間 $[L, R)$ を根として探索することで、「区間 $[L, R)$ の Binary Trie における探索」を実現することができます。

例えば次の問題を解くことができます.

【問題 1】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます.

Q 個のクエリに答えてください. クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R および, $0 \leq k < R - L$ を満たす整数 k が与えられるので, 区間 $[L, R)$ の中で (0-based index で) 昇順 k 番目の要素の値を答えてください.

この問題は, $O((N + Q)H)$ 時間で解くことができます. まず, $O(NH)$ 時間で Wavelet Matrix を構築します. クエリに答えるには, 例えば次のような再帰関数により区間 $[L, R)$ の Binary Trie を探索すればよいです.

```
int kth_smallest_rec(int h, int l, int r, int k) {
    if (h == 0) return 0;
    auto [l0, r0, l1, r1] = get_subtree_range(h - 1, l, r);
    int left_size = r0 - l0;
    if (k < left_size) {
        return kth_smallest_rec(h - 1, l0, r0, k);
    } else {
        return (1 << (h - 1)) + kth_smallest_rec(h - 1, l1, r1, k - left_size);
    }
}
```

このように Wavelet Matrix の探索では, 現在のノードは 3 つ組 (h, L, R) として表され, そのノードが高さ h での列 (5.1 節の図示参照) における区間 $[L, R)$ に対応することを意味します. この際, $[L, R)$ としては A 全体に対する Binary Trie のひとつのノード内の区間であるという条件が, 探索のどの時点においても成り立つことになります.

同様に次の問題を解くことができます.

【問題 2】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます.

Q 個のクエリに答えてください。クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R および、 $0 \leq a < b \leq 2^H$ を満たす整数 a, b が与えられるので、区間 $[L, R)$ の中で a 以上 b 未満の要素の個数を答えてください。

この問題は、 $O((N + Q)H)$ 時間で解くことができます。3.3 節の内容を思い出しながら実装を考えてみてください。

5.4. 実装例

Library Checker Range Kth Smallest への提出です。

- <https://judge.yosupo.jp/submission/353933>

6. 参考：0, 1 列の累積和 (rank) について

Wavelet Matrix では、長さ N の 0, 1 列に対して次の値が求められることを前提としています。

- $\text{rank}(i)$: 区間 $[0, i)$ に含まれる 1 の個数。

5 節の解説では累積和を用いることで、各層ごとに空間 $O(N)$ を使用して、rank クエリに $O(1)$ 時間で答えていました。

競技プログラミングでは、0, 1 列を bit vector として扱い、大幅に空間を削減しながら rank クエリに高速に答える次のような実装がよく使われています。

- 長さ N の 0, 1 列を、64 bit 整数 $\lceil N/64 \rceil$ 個として扱う。
- 各 $j = 0, 1, \dots, \lfloor N/64 \rfloor$ に対して、 $\text{rank}(64j)$ を求めて配列に格納しておく。
- $\text{rank}(i)$ のクエリには、 $i = 64j + k$ ($0 \leq k < 64$) としたときに、 $\text{rank}(64j)$ に端数 k ビット分の popcount を加えることで答える。

詳しくは次の実装例を参考にしてください。

- <https://judge.yosupo.jp/submission/353934>

この実装はワードに対する popcount を前提としています。これを $O(1)$ 時間であるとするかは計算量モデル次第ですが、いずれにせよ非常に高速です。

なお学術的には、bit vector に対して rank（場合によっては select）を高速に処理できるような補助情報を付与したデータ構造（完備辞書，fully indexable dictionary）が用いられることもあります。

7. 関連問題

1. https://judge.yosupo.jp/problem/range_kth_smallest
2. <https://codeforces.com/problemset/problem/323/C>
3. <https://yukicoder.me/problems/no/1332>
4. https://atcoder.jp/contests/abc239/tasks/abc239_e
5. <https://yukicoder.me/problems/no/924>
6. https://atcoder.jp/contests/arc214/tasks/arc214_e

▶ 問題について

8. まとめ

本講座では、Wavelet Matrix について、Binary Trie との関係を強調しながら、そのアイデア、実装方法、区間に対する Binary Trie としての探索方法を解説しました。さらに具体的に、区間内での昇順 k 番目の要素や、 a 以上 b 未満の要素の個数を求める問題が解けることを確認しました。

Wavelet Matrix はこれらの処理ができるだけでもかなり優秀なデータ構造ですが、

- 追加のデータを持たせることで要素の総和などを計算する。
- 座標圧縮などと組み合わせて 2 次元の点群に対する矩形クエリに答える。

などの応用も考えられます。どのような問題に応用できるかについては改めて、Wavelet Matrix（発展） で整理します。

また、Wavelet Matrix は適切に実装すれば、入力の列の大きさとほとんど同じ $NH(1 + o(1))$ ビットの空間で実装することができ、**簡潔データ構造** (succinct data structure) と

しても知られています。ただし、この空間計算量を達成するには、0, 1 列の累積和 (rank) を扱う部分について、本講座で述べた素朴な方法ではなく、簡潔ビットベクトル (succinct bit vector) などのより洗練された実装が必要です。このことは競技プログラミングでは重要となることがあまりないため、解説を省略します。

9. 参考文献

1. Roberto Grossi, Ankur Gupta, Jeffrey Scott Vitter. High-Order Entropy-Compressed Text Indexes. Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). pp. 841–850. 2003.
<https://pages.di.unipi.it/grossi/PAPERS/sodaconf03FINAL-LATEST.pdf>.
2. Francisco Claude, Gonzalo Navarro, Alberto Ordóñez Pereira. The wavelet matrix: An efficient wavelet tree for large alphabets. Information Systems. Vol. 47. pp. 15–32. 2015. DOI: 10.1016/j.is.2014.06.002.
<https://doi.org/10.1016/j.is.2014.06.002>. (Preprint)
<https://arxiv.org/abs/1405.1220>.
3. USACO Guide. Wavelet Tree. <https://usaco.guide/adv/wavelet>. (閲覧日: 2026-03-04).
4. Preferred Networks Tech Blog. ウェーブレット木の世界.
https://tech.preferred.jp/ja/blog/wavelettree_world/. (閲覧日: 2026-03-04).
5. Mitl_7. ウェーブレット行列 (wavelet matrix). <https://mitl-7.hatenablog.com/entry/2018/04/28/152259>. (閲覧日: 2026-03-04).
6. Robinson Castro, Nico Lehmann, Jorge Pérez, Bernardo Subercaseaux. Wavelet Trees for Competitive Programming. Olympiads in Informatics. Vol. 10. pp. 19–37. 2016. DOI: 10.15388/ioi.2016.02.
<https://doi.org/10.15388/ioi.2016.02>.
https://ioinformatics.org/journal/v10_2016_19_37.pdf.
7. Wikipedia. 簡潔データ構造. <https://ja.wikipedia.org/wiki/簡潔データ構造>. (閲覧日: 2026-04-16).

本講座のように、「Wavelet Matrix とは区間に対する Binary Trie を扱えるデータ構造である」という解説をしている日本語文献は珍しいと思います。興味があれば他の文献とも読み比べてみてください。

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)

